

Кодирование файла по шагам — учебное пособие

Это третья, самая подробная версия руководства по кодированию файла в поток DataShield — «для учащихся техникума»: каждый термин вводится с нуля, каждый шаг разобран на числах, с таблицами и сквозным примером. Формальная монография — `EncoderGuide.Academic.ru.md`, короткий инженерный пересказ — `EncoderGuide.Engineer.ru.md` (нумерация разделов у них совпадает между собой; здесь своя, «уроковая»). Общий обзор формата — в `AlgorithmGuide.ru.md` (разделы 2–4); обратный процесс — семейство `DecoderGuide.*`; сборка файла из принятого — семейство `AssemblyGuide.*`.

Прочитав пособие, вы сможете своими словами объяснить: во что превращается файл на выходе кодера, откуда берутся все числа (`N`, `M`, `T`, количество заголовков), как устроены пакеты и их хеши, зачем нужны ЕСС-тома и как они расставляются в потоке.

1. Словарь: слова, без которых дальше никуда

Термин	Что это на самом деле
Кодер	Класс <code>FileEncoder</code> : превращает файл в список 75-байтных пакетов.
Пакет	Ровно 75 байт: неделимая единица потока. Бывает двух видов — заголовок и сектор.
Заголовок	Пакет-«шапка» файла: 51 байт полей + 24 байта хеша <code>H5</code> .
Сектор	Пакет с порцией данных: 2 байта номера + 64 байта payload + 9 байт хеша <code>D3</code> .
Payload	64 байта содержимого: кусок файла (data-том) или избыточность (ЕСС-том).
<code>N</code>	Число data-томов: <code>N = max(1, ceil(FileSize / 64))</code> .
<code>M</code>	Число ЕСС-томов: <code>M = max(1, ceil(N * eccPercent / 100))</code> при <code>eccPercent ≥ 1</code> , иначе 0.
<code>T</code>	<code>T = N + M</code> — общее число секторов данных в потоке.
<code>H1</code> — <code>H4</code>	Поля заголовка: имя (14 байт), размер (3), SHA-256 файла (32), число ЕСС-томов (2).
<code>H5</code>	Хеш заголовка: <code>Trunc24(SHA-256(H1-H4))</code> — 24 байта, «сид» для хешей секторов.
<code>H3</code>	SHA-256 содержимого исходного файла — будущий арбитр сборки на стороне декодера.
<code>D1</code> — <code>D3</code>	Поля сектора: номер (2 байта), payload (64), хеш (9).
ЕСС	Избыточность: код Рида–Соломона над полем GF(2 ¹⁶).
Процент ЕСС (<code>eccPercent</code>)	Настройка: сколько избыточности добавлять. По умолчанию 10%.
Процент заголовков (<code>headerPercent</code>)	Настройка: как часто повторять заголовок в потоке. По умолчанию 3%, минимум 3 копии.
FEС-поток	Итоговый поток пакетов: Base64-текст (<code>.DataShield.txt</code>) или сырые байты (<code>.DataShield.bin</code>).

2. Что делает кодер: общая картина

Вход — содержимое файла (байты) и его имя. Выход — список пакетов по 75 байт, которые дальше превращаются в текст или бинарный файл. Конвейер по шагам:

файл

- └ 1. проверки: размер $\leq 16\,777\,215$ байт, проценты ≥ 0
- └ 2. имя файла $\rightarrow$ 14-байтное поле $H1$
- └ 3. SHA-256 содержимого $\rightarrow H3$
- └ 4. размерности: $N, M, T = N + M$ (проверка $T \leq 65\,535$)
- └ 5. нарезка: N payload по 64 байта (последний добивается нулями)
- └ 6. ECC: M избыточных payload (код Рида-Соломона)
- └ 7. заголовок: $H1-H4 \rightarrow 51$ байт, хеш $H5 \rightarrow$ пакет заголовка
- └ 8. секторы: T пакетов вида $D1+D2+D3$
- └ 9. расстановка: заголовки в начало, конец и равномерно между секторами
- └ 10. выход: `List<byte[]>` $\rightarrow$ Base64-текст или бинарный файл (`PacketIO`)

Два процента настраиваются в конструкторе: `new FileEncoder(eccPercent: 10, headerPercent: 3)`.

3. Ограничения формата — три жёстких потолка

Ограничение	Значение	Причина
Размер файла	$\leq 16\,777\,215$ байт (~16 МиБ)	поле <code>H2</code> занимает 3 байта
Всего томов <code>T = N + M</code>	$\leq 65\,535$	размер поля $GF(2^{16})$ у кода Рида-Соломона
Размер пакета	ровно 75 байт	формат: $2 + 64 + 9$ (сектор) или $51 + 24$ (заголовок)

Превышение любого — исключение `InvalidOperationException` с внятным сообщением; кодер никогда не выдаёт «частично правильный» поток.

4. Шаг первый: имя файла в 14 байт (поле `H1`)

Поле имени в заголовке — всего 14 ASCII-символов. Правила упаковки (`FileNameCodec.Pack`):

- Берётся только имя файла без каталога (`Path.GetFileName`).
- Имя режется по **первой** точке: слева — база, справа — расширение. Расширение сохраняется целиком, даже составное: `.tar.gz` останется `.tar.gz`.
- Квантуется (усекается) только база. Если усекли — в конец базы добавляется маркер `~`.
- Если на базу остаётся меньше 2 байт (1 символ + маркер) — имя не представимо, исключение.

Примеры:

Исходное имя	Упакованное H1	Что произошло
report.pdf	report.pdf	влезло целиком (10 символов)
verylongname.txt	verylongnam~.txt	база усечена до 10 + маркер
a.tar.gz	a.tar.gz	составное расширение сохранено
my.documents.2024.xls x	my.documents~.2024.xls x	режим по первой точке; всё после неё — расширение

Крайние случаи: точки нет или она первая — всё имя считается базой.

5. Шаг второй: SHA-256 содержимого (H3)

От содержимого файла считается обычный SHA-256 — 32 байта. Это самый важный груз, который кодер кладёт в заголовок: именно по этому хешу декодер позже докажет, что собрал файл бит-в-бит. Пока файл не собран на той стороне, H3 — просто число; объяснение его роли — в [AssemblyGuide.Tutorial.ru.md](#).

6. Шаг третий: размерности N, M, T

Три формулы, которые надо уметь считать руками:

$N = \max(1, \text{ceil}(\text{FileSize} / 64))$	— data-тома
$M = \max(1, \text{ceil}(N \cdot \text{eccPercent} / 100))$	— ECC-тома (0, если eccPercent = 0)
$T = N + M$	— всего секторов

Таблица для eccPercent = 10:

FileSize, байт	N	M	T
1	1	1	2
64	1	1	2
65	2	1	3
1000	16	2	18
100 000	1563	157	1720

Обратите внимание на маленькие файлы: даже файлу в 1 байт нужен один data-том, а при 10% ECC минимум один ECC-том — избыточность 6400%. Это осознанно: формула $\max(1, \dots)$ гарантирует, что ECC-режим действительно восстанавливает потери даже на крошечных файлах.

7. Шаг четвёртый: нарезка на payload

Файл режется подряд на куски по 64 байта. Всего кусков N . Последний кусок — особенный: если файл не кратен 64, в нём реальных байтов меньше, и хвост добивается **нулями** до полных 64. Пример: файл 1000 байт → 15 полных кусков + 16-й кусок, где 40 байт данных и 24 нуля. Эти нули декодер позже отрежет — размер файла-то известен из заголовка ($H2$).

8. Шаг пятый: ЕСС-тома — избыточность Рида-Соломона

Если $M > 0$, кодер считает M избыточных payload по правилам стирающего кода Рида-Соломона над полем $GF(2^{16})$. Что это значит «на пальцах»:

- Один символ поля — 2 байта. Payload 64 байта = **32 символа**, и все 32 позиции обрабатываются независимо, столбец за столбцом.
- Для каждой позиции собирается «столбец»: по одному символу из каждого data-тома. Код достраивает в конец столбца M ЕСС-символов — линейных комбинаций data-символов над $GF(2^{16})$.
- Собранные ЕСС-символы раскладываются обратно по своим ЕСС-томам. Итог: M новых 64-байтных payload.

Зачем это нужно: если при передаче потеряются любые $k \leq M$ data-томов, декодер восстановит их по оставшимся data-томам и ЕСС-томам. Условие восстановления простое: **число потерянных data-томов $\leq$ числу доступных ЕСС-томов**. Потерянные ЕСС-тома вообще не мешают — восстановление их просто не использует.

Граничное условие поля: $N + M \leq 65\,535$. Сама математика $GF(2^{16})$ живёт в эталонном модуле `RsRaid16` и не изменяется.

9. Шаг шестой: заголовочный пакет и хеш $H5$

Заголовок собирается из полей:

Поле	Размер	Смещение	Содержимое
$H1$ имя	14	0	упакованное имя (раздел 4)
$H2$ размер	3	14	FileSize, little-endian
$H3$ SHA-256	32	17	хеш содержимого
$H4$ ЕСС-томов	2	49	M, little-endian

Итого 51 байт. Дальше считается $H5 = \text{Trunc24}(\text{SHA-256}(\text{эти 51 байт}))$ — берётся SHA-256 и от него остаются **первые 24 байта** (192 бита). Заголовочный пакет = 51 байт полей + 24 байта $H5$ = те же 75 байт, что и у сектора.

Зачем $H5$: это одновременно контроль целостности заголовка (декодер проверяет его автономно, без каких-либо ключей и внешних данных) и «сид» — ключ, которым подписываются все секторы этого файла (раздел 10).

10. Шаг седьмой: секторы данных (D1 + D2 + D3)

Каждый из T payload (N data + M ECC) превращается в сектор:

Поле	Размер	Смещение	Содержимое
D1 номер	2	0	0 ... T-1, little-endian
D2 payload	64	2	64 байта содержимого
D3 хеш	9	66	Trunc9 (SHA-256 (H5 D1 D2))

D3 — усе́ченный до 9 байт (72 бита) SHA-256 от склейки: 24 байта H5 + 66 байт содержимого сектора. Ключевой трюк: **без H5 этот хеш проверить нельзя**. Сектор намертво привязан к своему файлу: чужой пакет, даже с правильным номером и осмысленным содержимым, проверку не пройдёт. Итог: T секторов по 75 байт.

11. Шаг восьмой: сколько заголовков и куда они встают

Декодеру заголовок нужен, чтобы вообще начать понимать секторы (нужен H5). Поэтому заголовок повторяется:

```
число копий = max(3, ceil(T · headerPercent / 100))
```

Минимум 3 — всегда: начало, середина и конец потока. Расстановка (ArrangePackets):

- 1. Первый пакет потока — заголовок.
- 2. Последний пакет потока — заголовок.
- 3. Промежуточные копии — равномерно: интервал = max(1, T / (промежуточные + 1)) , заголовок вставляется после каждого интервал -го сектора, пока копии не кончатся.

Примеры:

T	headerPercent	Копий	Интервал	Поток (H — заголовок)
2	3	3	—	H D0 D1 H
18	3	3	9	H D0...D8 H D9...D17 H
200	3	6	40	H D0...D39 H D40...D79 H ... H D160...D199 H

Почему это важно: даже если из середины потока вырезали кусок, почти наверняка уцелеет хотя бы одна копия заголовка, и декодер сможет опознать файл.

12. Шаг девятый: выходной формат

Кодер отдаёт List<byte[]> — упорядоченные 75-байтные пакеты. Дальше два формата на выбор (OutputFormat):

Формат	Расширение	Что внутри
Base64	<code>.DataShield.txt</code>	текст: по строке на пакет, ровно 100 символов Base64 в строке; опционально декоративные строки-обрамления <code>>[имя] [размер] [SHA-256:hex]</code> в начале и <code><[...]</code> в конце
Binary	<code>.DataShield.bin</code>	сырые пакеты подряд, без разделителей: $75 \times \text{число пакетов байт}$

Почему Base64-строка — ровно 100 символов: 75 байт = 25 троек, каждая тройка кодируется в 4 символа; 75 делится на 3 нацело, поэтому символов `=` (добивки) не бывает никогда.

Декоративные строки — только для человека: декодер их игнорирует (не-Base64 символы отбрасывает фильтр).

Запись в файл — через `PacketIO.WriteFile`; он же читает обратно (`ScanFile`), определяя формат по расширению.

13. Прогресс и отмена

`Encode` принимает `IProgress<CodecProgress>` и `CancellationToken`. Глобальная шкала 0...100 делится на фазы:

Фаза	Диапазон	Что происходит
Подготовка данных	0–10	проверки, SHA-256, нарезка
ЕСС-кодирование	10–75	код Риды–Соломона (самая тяжёлая часть)
Формирование пакетов	75–100	сборка <code>T</code> секторов
Готово	100	<code>Done</code>

Отмена проверяется между фазами и внутри ЕСС-цикла; при отмене летит `OperationCanceledException` — недоделанный поток не возвращается никогда.

14. Статистика кодирования (`EncodeStats`)

`EncodeWithStats` возвращает вместе с пакетами сводку: размер файла, SHA-256, `N`, `M`, общее число пакетов и сколько среди них копий заголовка. Удобно для интерфейса и логов: можно показать пользователю «18 секторов + 3 заголовка = 21 пакет».

15. Гигиена памяти

После сборки пакетов кодер **обнуляет** промежуточные буферы: все `N` data-payload, все `M` ЕСС-payload и байты заголовка. Сами пакеты, разумеется, живут — они и есть результат. Смысл вайпа: не оставлять в памяти лишних копий содержимого файла дольше необходимого.

16. Поточковый вход

Кроме байтового массива, кодер принимает `Stream` — читает его целиком (SHA-256 и RS требуют полного содержимого). Позиционируемые потоки читаются сразу с проверкой остатка; непозиционируемые копируются через промежуточный буфер. Проверка « ≤ 16 МиБ» работает в обоих случаях. Поток кодер не закрывает.

17. Сквозной пример: файл 1000 байт, 10% ЕСС, 3% заголовков

Считаем всё по шагам:

1. Размер $1000 \leq 16\,777\,215$ — ок.
2. `N = ceil(1000/64) = 16` ($15 \times 64 = 960$, нужен 16-й кусок).
3. `M = max(1, ceil(16 · 10 / 100)) = max(1, 2) = 2`.
4. `T = 18 ≤ 65 535` — ок.
5. Payload: 15 полных по 64 байта + кусок №15 из 40 байт данных и 24 нулей.
6. ЕСС: 2 избыточных payload по правилам раздела 8.
7. Заголовок: `H1` = упакованное имя, `H2` = 1000, `H3` = SHA-256 файла, `H4` = 2; `H5` = `Trunc24(SHA-256(H1–H4))`.
8. Секторы: №0–15 — data, №16–17 — ЕСС, у каждого свой `D3`.
9. Заголовков: `max(3, ceil(18 · 3 / 100)) = max(3, 1) = 3`; интервал `= 18 / (1+1) = 9` → вставка после сектора №8.
10. Итоговый поток: `H D0...D8 H D9...D17 H` — **20 пакетов**.

Размеры выхода: бинарный поток $20 \times 75 = \mathbf{1500 \text{ байт}}$ (150% исходного); Base64-текст 20 строк $\times$ 100 символов = **2000 символов** плюс переводы строк. Расширение избыточности на больших файлах стремится к $\approx 132\%$ в бинарном виде ($75/64 \times 1.10 \times 1.03$) и $\approx 176\%$ в Base64.

18. Краевые случаи

Случай	Что происходит
Пустой файл (0 байт)	$N = \max(1, 0) = 1$: один data-том из одних нулей; файл «существует» и собирается
<code>eccPercent = 0</code>	$M = 0$, ECC-фаза пропускается; восстановление потерь невозможно, только повторы
Файл > 16 МиБ	<code>InvalidOperationException</code> немедленно
$N + M > 65\,535$	<code>InvalidOperationException</code> с подсказкой уменьшить размер или процент
Очень длинное расширение (> 12 байт)	базе не остаётся бюджета → <code>InvalidOperationException</code> от упаковки имени
Отрицательные проценты	<code>ArgumentOutOfRangeException</code> из конструктора

19. Что гарантируется

- Поток либо собран целиком и корректно, либо кодер бросил исключение — «частичного» выхода не бывает.
- `H3` в заголовке — точный SHA-256 содержимого: декодер получит безупречного арбитра подлинности.
- Каждый сектор подписан сидом `H5` своего файла: чужие пакеты в этот файл не «приклеятся».
- При $M \geq k$ потеря любых `k` data-томов восстановима (в рамках корректности RS над $GF(2^{16})$).
- Минимум 3 копии заголовка: начало, конец и равномерно середина.

20. Контрольные вопросы

- Почему пакет Base64 — ровно 100 символов и без `=`? (раздел 12)
- Из каких полей и как выводится `T`? (разделы 6, 8)
- Почему маленькие файлы получают гигантский процент избыточности и почему это правильно? (раздел 6)
- Чем `H5` отличается от `H3` и почему оба нужны? (разделы 5, 9)
- Почему сектор нельзя проверить без заголовка своего файла? (раздел 10)
- Сколько копий заголовка будет в потоке с `T = 333` при 3%? (раздел 11: `max(3, ceil(9.99)) = 10`)
- Что произойдёт с файлом в 1 байт при `eccPercent = 10`? (раздел 18)
- Зачем кодер обнуляет промежуточные буферы? (раздел 15)
- Какие три жёстких потолка формата и откуда они берутся? (раздел 3)
- Во что превратится файл 1000 байт на выходе? (раздел 17)